

docs.google.com – 全画面表示を終了するには、Esc を押します

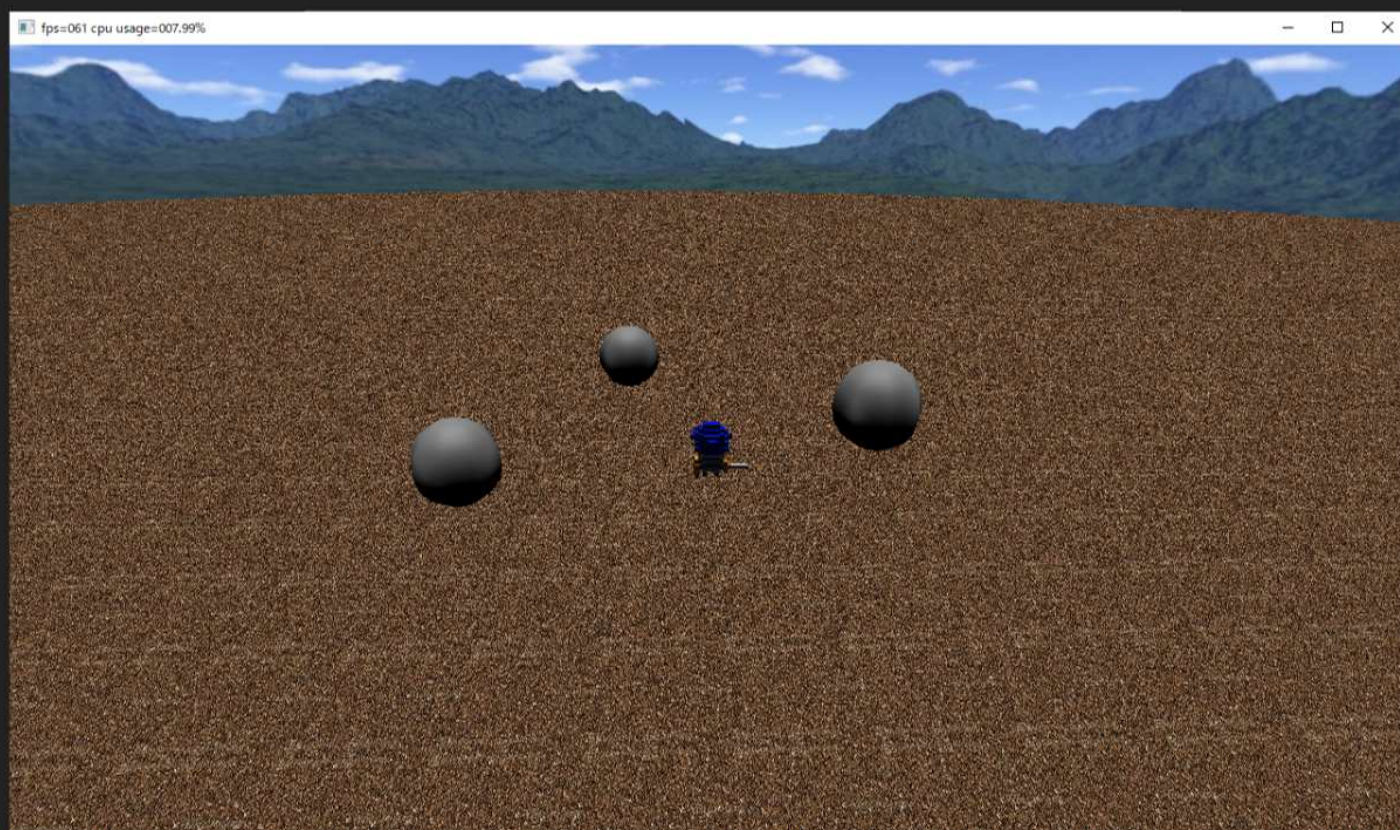
TL1

02_03.無効フラグ

日本工学院専門学校
デザインカレッジ

実行結果

日本工学院専門学校
デザインカレッジ



Blender側で無効フラグを設定したオブジェクトがゲームに配置されなくなった。

オブジェクトの走査

日本工学院専門学校
デザインカレッジ

レベルローダー

```
// "objects"の全オブジェクトを走査
for (nlohmann::json& object : deserialized["objects"]) {
    assert(object.contains("type"));

    if (object.contains("無効オプション")) {
        // 有効無効フラグ
        bool disabled = object["無効オプション"].get<bool>();
        if (disabled) {
            // 配置しない (スキップ)
            continue;
        }
    }

    // 種別を取得
    std::string type = object["type"].get<std::string>();

    種類ごとの処理

    再帰処理
}
```

ゲーム側（ローダー）の処理。

配置データを元に実際に配置していく
処理のところで、無効フラグを検出したら
配置を飛ばす。

これによって、無効フラグが存在しない場
合とFalseの場合は配置され、
無効フラグがTrueの時だけ配置される。

ゲーム側の対応

エクスポーター

日本工学院専門学校
デザインカレッジ

export_scene.py

```
+ #カスタムプロパティ'無効オプション'  
if "無効オプション" in object:  
    json_object["無効オプション"] = object["無効オプション"]  
  
#カスタムプロパティ'file_name'  
...省略...  
  
#カスタムプロパティ'collider'  
...省略...
```

無効フラグの実装として

1. 無効フラグがTrueならJsonファイルにそのオブジェクトを出力しない
2. 無効フラグの情報ごとJsonファイルに出力する

のパターンがありえる。

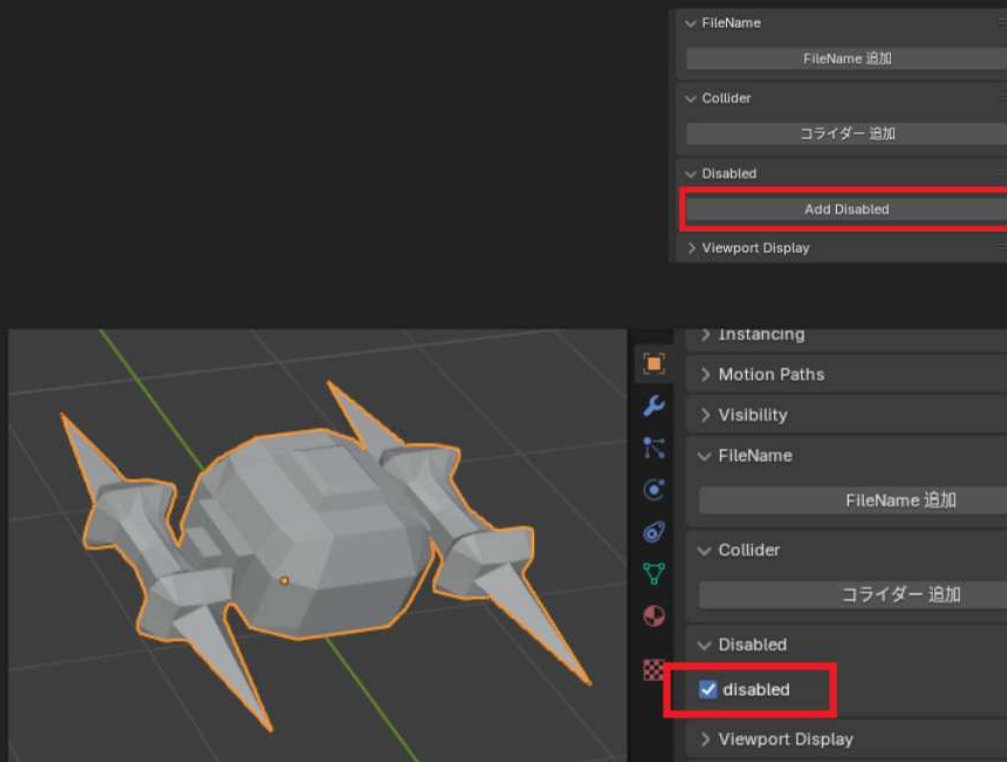
どちらも一長一短あるが、今回は2.の方法を採用する。

1.の場合はゲーム側ではオブジェクトを認識することすらできないので、ゲーム側での追加作業は不要。こちらの方が実装としては簡単かもしれない。

エクスポーター

実行結果

日本工学院専門学校
デザインカレッジ



以前に、ファイル名やコライダーをパネルに追加したが、それと同様に無効フラグを追加した。

無効フラグがない時は追加用のボタンが表示され、無効フラグがあればフラグを表示する。

左図の通り、bool 型のカスタムプロパティを Panel で表示すれば、自動的にチェックボックス表示になる。

クラスリストに追加

日本工学院専門学校
デザインカレッジ

```
__init__.py
# Blenderに登録するクラスリスト
classes = (
    ...省略...
    無効オプション追加オペレータ,
    無効オプションパネル,
)
```

importが済んだので、

`register()` と `unregister()`

で参照するため、Blenderに登録するクラスリスト
に今回自作したクラスを登録する。

これによってBlenderが自作クラスを認識してくれる
状態になる。

import

日本工学院専門学校
デザインカレッジ

`__init__.py`

自分で考えよう

モジュールを分割しているが、
Blenderに自作クラスを登録する作業があるので
`__init__.py` に `import` が必要。

分からなければ[02_02]を参照しよう。

`__init__.py`

パネルクラス

日本工学院専門学校
デザインカレッジ

```
disabled.py
#パネル 無効オプション
class 無効オプションパネル(bpy.types.Panel):
    ...省略...

    # サブメニューの描画
    def draw(self, context):
        新たなカスタムプロパティ = True

        return {'FINISHED'}
```

先ほどのオペレータとセットで使うので、同じスクリプトファイルに記述してもよいだろう。

続いて、

- カスタムプロパティがあればそれを表示し、
- なければカスタムプロパティ追加用のボタン（オペレータ）を表示する

ためのパネルを作成する。

こちらも既にやったことがあるので[01_08]を参照。

オペレータクラス

日本工学院専門学校
デザインカレッジ

```
disabled.py
#オペレータ 無効オプションを追加する
class 無効オプション追加オペレータ(bpy.types.Operator):
    ...省略...

    def execute(self, context):
        新たなカスタムプロパティ = True

        return {'FINISHED'}
```

新クラスは新しくモジュール用の
スクリプトファイルを作成し、記述する。

チェックボックスは、bool型のカスタムプロパティを
パネルで表示すればチェックボックスになる。

ということで、無効フラグを意味する変数を
カスタムプロパティとして設定したいので

[01_05]や[01_08]を思い出しつつ
カスタムプロパティを追加するための
オペレータを作成する。

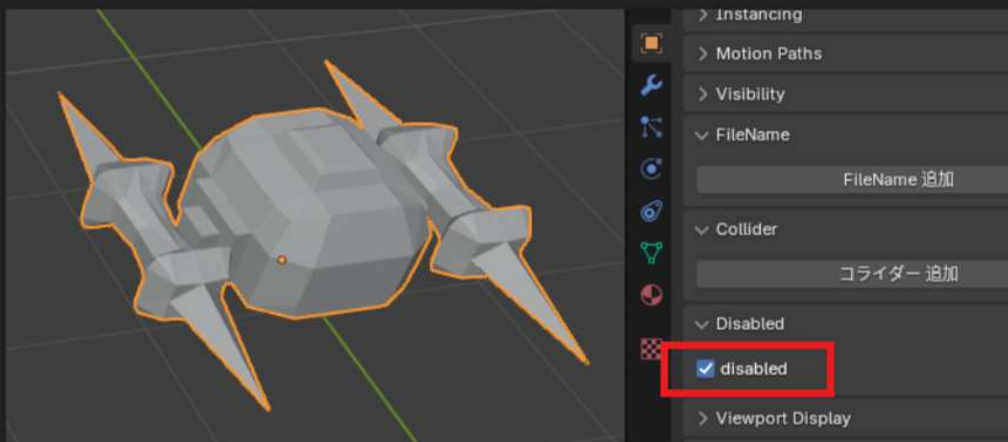
python の bool 型の値は
True / False のいずれかである。

モジュール

今回の目標

日本工学院専門学校
デザインカレッジ

- Blender上で配置しているオブジェクトについて、ゲームに出す／出さないのフラグを設定できる。



Blender上で配置しているオブジェクトについて、ゲームに出す／出さないのフラグを設定できる。

開発の都合上、オブジェクトを一時的に消したい場合などに使える。

TL1

02_03.無効フラグ